

Document 4 — Software Architecture & Development Structure

OpenHealth

Complete Codebase Architecture, Backend Structure, API Flow, AI Services, Security & Development Organization

Document Purpose

This document defines how the OpenHealth application should be organized at the **codebase and engineering level**.

It translates the Product Blueprint, Technical Implementation Plan, and Frontend Blueprint into a practical structure that developers can directly implement.

The architecture uses:

- **React**
- **Tailwind CSS**
- **React Router**
- **Node.js**
- **Express.js**
- **Supabase PostgreSQL**
- **Supabase Auth**
- **Supabase Storage**
- **Python + FastAPI for AI/document processing**
- **OCR**
- **LLM APIs**

TypeScript is not used. The frontend and JavaScript backend use `.jsx` and `.js`.

1. Architecture Philosophy

OpenHealth should be scalable enough to support future healthcare integrations but simple enough to build rapidly during the hackathon.

The most important principle is:

Separate presentation, API handling, business logic, AI processing, and data access.

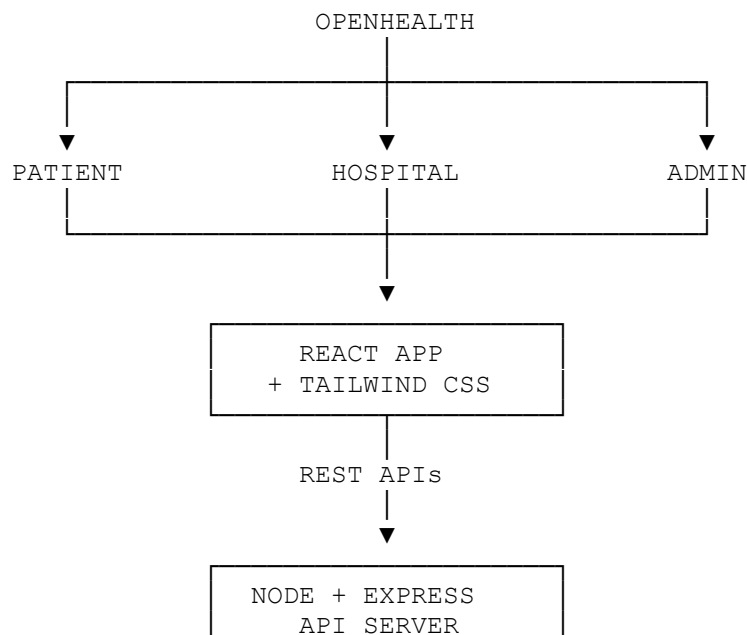
The basic architecture is:

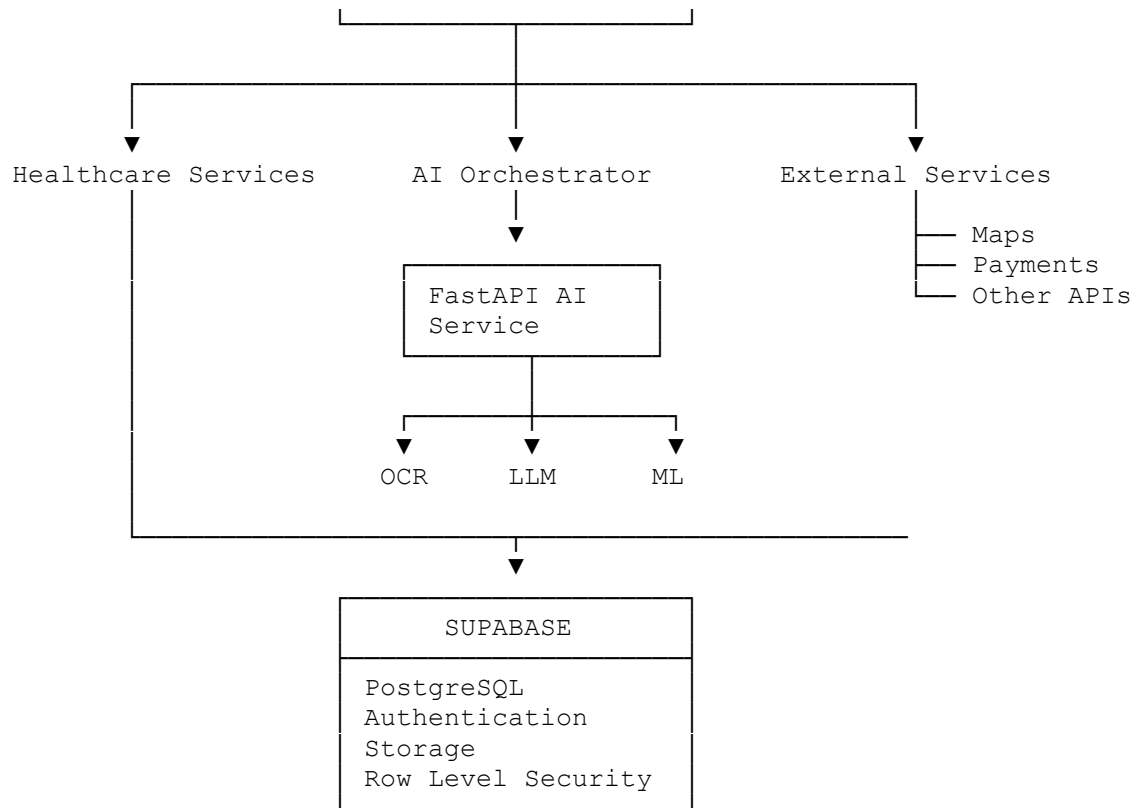
```
graph TD
  USER --> REACT_TAILWIND[REACT + TAILWIND]
  REACT_TAILWIND --> API_SERVICE[API SERVICE]
  API_SERVICE --> EXPRESS_ROUTE[EXPRESS ROUTE]
  EXPRESS_ROUTE --> CONTROLLER[CONTROLLER]
  CONTROLLER --> BUSINESS_SERVICE[BUSINESS SERVICE]
  BUSINESS_SERVICE --> DB_SERVICE[DATABASE / EXTERNAL SERVICE]
```

AI-heavy features additionally use:

```
graph TD
  EXPRESS_SERVICE[EXPRESS SERVICE] --> FASTAPI_AI_SERVICE[FASTAPI AI SERVICE]
  FASTAPI_AI_SERVICE --> OCR_AI_ML[OCR / AI / ML]
  OCR_AI_ML --> STRUCTURED_RESULT[STRUCTURED RESULT]
  STRUCTURED_RESULT --> EXPRESS[EXPRESS]
  EXPRESS --> DATABASE[DATABASE]
  DATABASE --> REACT[REACT]
```

2. Complete System Architecture





3. Monorepo Structure

The complete project should use a monorepo-style structure.

```
OpenHealth/  
├── client/  
├── server/  
├── ai-service/  
├── database/  
├── docs/  
├── scripts/  
├── .env.example  
├── .gitignore  
├── package.json  
└── README.md
```

This keeps major application layers independently understandable.

4. Frontend Architecture

```
client/
├── public/
│   ├── logo.svg
│   ├── favicon.svg
│   └── images/
├── src/
│   ├── assets/
│   │   ├── images/
│   │   ├── icons/
│   │   └── illustrations/
│   ├── components/
│   ├── pages/
│   ├── services/
│   ├── hooks/
│   ├── context/
│   ├── routes/
│   ├── lib/
│   ├── constants/
│   ├── utils/
│   ├── App.jsx
│   ├── main.jsx
│   └── index.css
├── .env
├── .env.example
├── package.json
└── vite.config.js
```

5. Frontend Components

Components should be grouped by domain rather than placing every component in one folder.

```
components/
├── common/
├── layout/
└── dashboard/
```

- search/
- hospitals/
- doctors/
- treatments/
- beds/
- emergency/
- bookings/
- documents/
- reports/
- bills/
- costs/
- insurance/
- schemes/
- transparency/
- admin/

6. Common Components

- common/
 - Button.jsx
 - Input.jsx
 - Textarea.jsx
 - Select.jsx
 - Checkbox.jsx
 - Radio.jsx
 - Switch.jsx
 - Modal.jsx
 - Drawer.jsx
 - Dropdown.jsx
 - Tooltip.jsx
 - Toast.jsx
 - Loader.jsx
 - Skeleton.jsx
 - EmptyState.jsx
 - ErrorState.jsx
 - ConfirmDialog.jsx
 - Badge.jsx
 - Avatar.jsx
 - FileUploader.jsx

These should remain generic and reusable.

7. Layout Components

```
layout/  
├─ AppLayout.jsx  
├─ Sidebar.jsx  
├─ Navbar.jsx  
├─ MobileNavbar.jsx  
├─ Footer.jsx  
├─ Breadcrumb.jsx  
└─ ProtectedRoute.jsx
```

Responsibility

AppLayout.jsx

Controls the overall authenticated application shell.

Sidebar.jsx

Controls desktop navigation.

Navbar.jsx

Controls search, notifications, location, and profile.

ProtectedRoute.jsx

Prevents unauthenticated access.

8. Search Components

```
search/  
├─ SearchBar.jsx  
├─ SearchTypeSelector.jsx  
├─ SearchFilters.jsx  
├─ SearchResults.jsx  
├─ SearchResultCard.jsx  
├─ SearchSuggestions.jsx  
├─ MapView.jsx  
└─ LocationSelector.jsx
```

9. Hospital Components

hospitals/

- HospitalCard.jsx
- HospitalHeader.jsx
- HospitalProfile.jsx
- HospitalVerificationBadge.jsx
- HospitalFacilities.jsx
- HospitalDepartments.jsx
- HospitalDoctors.jsx
- HospitalTreatments.jsx
- HospitalPackages.jsx
- HospitalAvailability.jsx
- HospitalTransparency.jsx
- HospitalComparison.jsx

10. Doctor Components

doctors/

- DoctorCard.jsx
- DoctorProfile.jsx
- DoctorFilters.jsx
- DoctorSearch.jsx
- DoctorVerificationBadge.jsx

11. Bed Components

beds/

- BedAvailabilityCard.jsx
- BedSearch.jsx
- BedTypeSelector.jsx
- BedStatusBadge.jsx
- BedAvailabilityTable.jsx
- ReservationCard.jsx
- AvailabilityFreshness.jsx

12. Emergency Components

emergency/

- EmergencyBanner.jsx
- EmergencySearch.jsx
- EmergencyHospitalCard.jsx
- EmergencyBedResult.jsx
- EmergencyReservation.jsx
- EmergencyLocation.jsx

13. Document Components

documents/

- DocumentUploader.jsx

- DocumentCard.jsx
- DocumentPreview.jsx
- ProcessingStatus.jsx
- ExtractedInformation.jsx
- AIExplanation.jsx

14. Bill Components

bills/

- BillUploader.jsx
- BillSummary.jsx
- BillLineItems.jsx
- BillBreakdownChart.jsx
- BillAnomalyCard.jsx
- BillShockIndex.jsx
- EstimateVsActual.jsx
- BillAnalysis.jsx

15. Cost Components

costs/

- CostPredictionForm.jsx
- CostRangeCard.jsx
- CostBreakdown.jsx
- CostConfidence.jsx
- CostFactors.jsx

16. Insurance Components

insurance/

- InsuranceForm.jsx
- CoverageCard.jsx
- CoverageBreakdown.jsx
- CopayCard.jsx
- InsuranceStatus.jsx

17. Transparency Components

transparency/

- TransparencyScore.jsx
- TransparencyBreakdown.jsx
- TransparencyBadge.jsx
- BillingConsistency.jsx
- TransparencyHistory.jsx

18. Patient Pages

```
pages/patient/  
├── Dashboard.jsx  
├── Search.jsx  
├── SearchResults.jsx  
├── HospitalDetails.jsx  
├── CompareHospitals.jsx  
├── Doctors.jsx  
├── Treatments.jsx  
├── Beds.jsx  
├── Emergency.jsx  
├── BedReservation.jsx  
├── CostPrediction.jsx  
├── InsuranceChecker.jsx  
├── SchemeChecker.jsx  
├── Documents.jsx  
├── ReportAnalysis.jsx  
├── Bills.jsx  
├── BillAnalysis.jsx  
├── Bookings.jsx  
├── BookingDetails.jsx  
├── SavedHospitals.jsx  
└── Settings.jsx
```

19. Hospital Pages

```
pages/hospital/  
├── Dashboard.jsx  
├── Profile.jsx  
├── Beds.jsx  
├── Doctors.jsx  
├── Departments.jsx  
├── Treatments.jsx  
├── Packages.jsx  
├── Bookings.jsx  
├── Analytics.jsx  
└── Transparency.jsx
```

20. Admin Pages

```
pages/admin/  
├── Dashboard.jsx  
├── Users.jsx  
├── Hospitals.jsx  
├── Doctors.jsx  
├── Verification.jsx  
├── Schemes.jsx  
├── Insurance.jsx  
├── Analytics.jsx  
└── AuditLogs.jsx
```

21. Public Pages

```
pages/public/
├── Landing.jsx
├── HowItWorks.jsx
├── HospitalDiscovery.jsx
├── HospitalPublicProfile.jsx
└── EmergencyEntry.jsx
```

22. Authentication Pages

```
pages/auth/
├── Login.jsx
├── Signup.jsx
├── ForgotPassword.jsx
├── RoleSelection.jsx
└── Onboarding.jsx
```

23. Frontend Services

All backend communication should happen through service modules.

```
services/
├── api.js
├── authService.js
├── userService.js
├── hospitalService.js
├── doctorService.js
├── departmentService.js
├── treatmentService.js
├── packageService.js
├── bedService.js
├── bookingService.js
├── searchService.js
├── documentService.js
├── reportService.js
├── billService.js
├── costService.js
├── insuranceService.js
├── schemeService.js
├── transparencyService.js
├── emergencyService.js
└── adminService.js
```

24. Example Frontend Service

```
// hospitalService.js

import api from "../api";

export async function getHospitals(params) {
  const response = await api.get("/hospitals", { params });
  return response.data;
}

export async function getHospital(id) {
  const response = await api.get(`/hospitals/${id}`);
  return response.data;
}

export async function getHospitalAvailability(id) {
  const response = await api.get(`/hospitals/${id}/availability`);
  return response.data;
}
```

Components should call these services rather than constructing API requests themselves.

25. Hooks

```
hooks/
├── useAuth.js
├── useUser.js
├── useHospitals.js
├── useHospital.js
├── useSearch.js
├── useBeds.js
├── useBookings.js
├── useDocuments.js
├── useBills.js
├── useCostPrediction.js
├── useInsurance.js
└── useNotifications.js
```

Hooks encapsulate reusable frontend logic.

26. Context

Use React Context only for state that genuinely needs global access.

```
context/
├── AuthContext.jsx
├── UserContext.jsx
└── NotificationContext.jsx
```

Avoid unnecessary global state.

27. Routing

```
routes/
├── AppRoutes.jsx
├── PublicRoutes.jsx
├── PatientRoutes.jsx
├── HospitalRoutes.jsx
├── AdminRoutes.jsx
└── ProtectedRoute.jsx
```

The router should enforce role-based navigation at the UI level, while the backend independently verifies authorization.

28. Backend Architecture

The backend follows:

```
Routes
  ↓
Controllers
  ↓
Services
  ↓
Database / External APIs
```

Recommended structure:

```
server/
├── src/
│   ├── config/
│   ├── controllers/
│   ├── routes/
│   ├── services/
│   ├── middleware/
│   ├── validators/
│   ├── constants/
│   ├── utils/
│   ├── app.js
│   └── server.js
├── .env
├── .env.example
├── package.json
└── README.md
```

29. Backend Configuration

```
config/  
├── db.js  
├── env.js  
├── ai.js  
├── storage.js  
├── maps.js  
└── payment.js
```

Responsibilities

db.js

Database connection/client.

env.js

Environment configuration and validation.

ai.js

AI service configuration.

storage.js

Supabase Storage configuration.

maps.js

Maps/geolocation configuration.

payment.js

Payment gateway configuration.

30. Controllers

```
controllers/  
├── authController.js  
├── userController.js  
├── hospitalController.js  
├── doctorController.js  
├── departmentController.js  
├── treatmentController.js  
├── packageController.js  
├── bedController.js  
└── bookingController.js
```

```
|— searchController.js
|— documentController.js
|— reportController.js
|— billController.js
|— costController.js
|— insuranceController.js
|— schemeController.js
|— transparencyController.js
|— emergencyController.js
|— adminController.js
```

Controllers should remain thin.

They should:

1. receive the request
2. validate/request parsed input
3. call the relevant service
4. return the response

Business logic should remain in services.

31. Backend Services

```
services/
|
|— search/
|   |— searchService.js
|   |— matchingService.js
|   |— rankingService.js
|
|— hospitals/
|   |— hospitalService.js
|   |— verificationService.js
|
|— doctors/
|   |— doctorService.js
|
|— treatments/
|   |— treatmentService.js
|
|— packages/
|   |— packageService.js
|
|— beds/
|   |— bedService.js
|   |— availabilityService.js
|
|— bookings/
|   |— bookingService.js
|   |— reservationService.js
```

```
├── ai/
│   ├── aiClient.js
│   ├── promptBuilder.js
│   ├── reportAnalyzer.js
│   ├── billAnalyzer.js
│   ├── careMatcher.js
│   └── explanationService.js
├── documents/
│   ├── documentService.js
│   └── piiMaskingService.js
├── bills/
│   ├── billService.js
│   ├── billAnalysisService.js
│   └── billShockService.js
├── costs/
│   ├── costPredictionService.js
│   └── pricingService.js
├── insurance/
│   └── insuranceService.js
├── schemes/
│   └── schemeService.js
├── transparency/
│   ├── transparencyService.js
│   └── scoreService.js
```

32. Backend Routes

```
routes/
├── authRoutes.js
├── userRoutes.js
├── hospitalRoutes.js
├── doctorRoutes.js
├── departmentRoutes.js
├── treatmentRoutes.js
├── packageRoutes.js
├── bedRoutes.js
├── bookingRoutes.js
├── searchRoutes.js
├── documentRoutes.js
├── reportRoutes.js
├── billRoutes.js
├── costRoutes.js
├── insuranceRoutes.js
├── schemeRoutes.js
├── transparencyRoutes.js
├── emergencyRoutes.js
└── adminRoutes.js
```

33. Middleware

```
middleware/  
├── authMiddleware.js  
├── roleMiddleware.js  
├── errorMiddleware.js  
├── validationMiddleware.js  
├── uploadMiddleware.js  
├── rateLimitMiddleware.js  
└── requestLogger.js
```

Auth middleware

Verifies authentication.

Role middleware

Checks role permissions.

Validation middleware

Validates request payloads.

Upload middleware

Validates uploaded files.

Rate limiting

Protects sensitive APIs and AI endpoints.

34. Validators

```
validators/  
├── authValidator.js  
├── userValidator.js  
├── hospitalValidator.js  
├── doctorValidator.js  
├── treatmentValidator.js  
├── packageValidator.js  
├── bedValidator.js  
├── bookingValidator.js  
├── documentValidator.js  
├── billValidator.js  
└── insuranceValidator.js
```

35. Backend Constants

```
constants/  
├── userRoles.js  
├── bedStatuses.js  
├── bookingStatuses.js  
├── documentTypes.js  
├── hospitalStatuses.js  
├── verificationStatuses.js  
├── transparencyWeights.js  
└── errorCodes.js
```

Avoid hard-coding business values throughout the application.

36. Backend Utilities

```
utils/  
├── response.js  
├── errors.js  
├── logger.js  
├── pagination.js  
├── distance.js  
└── validators.js
```

37. AI Service Architecture

The AI service should be independently deployable.

```
ai-service/  
├── app/  
│   ├── main.py  
│   └── api/  
│       ├── report_routes.py  
│       ├── bill_routes.py  
│       ├── matching_routes.py  
│       └── health_routes.py  
│   └── services/  
│       ├── ocr_service.py  
│       ├── report_analyzer.py  
│       ├── bill_analyzer.py  
│       ├── matching_service.py  
│       ├── llm_service.py  
│       └── pii_service.py  
└── prompts/  
    └── report_prompt.py
```

```
├── bill_prompt.py
├── explanation_prompt.py
├── schemas/
│   ├── report_schema.py
│   ├── bill_schema.py
│   └── matching_schema.py
├── utils/
│   ├── logging.py
│   └── validation.py
├── requirements.txt
├── .env.example
└── README.md
```

38. FastAPI Endpoints

```
POST /ai/report/analyze
POST /ai/bill/analyze
POST /ai/match/care
POST /ai/explain
GET /ai/health
```

39. AI Report Processing

```
React
┆
┆ Express
┆
┆ Document Storage
┆
┆ FastAPI
┆
┆ OCR
┆
┆ Text Cleaning
┆
┆ Information Extraction
┆
┆ LLM Interpretation
┆
┆ Structured JSON
┆
┆ Express
┆
┆ Database
┆
┆ React
```

The AI response should be structured rather than returning only a free-form paragraph.

40. AI Bill Processing

```
Bill
↓
OCR
↓
Text Extraction
↓
Table / Line Item Detection
↓
Category Classification
↓
Amount Extraction
↓
Anomaly Analysis
↓
Structured Result
↓
Bill Shock Calculation
↓
Database
```

41. AI Care Matching

```
Patient Input
↓
Symptom / Treatment Extraction
↓
Department Mapping
↓
Treatment Mapping
↓
Hospital Capability Matching
↓
Availability
↓
Location
↓
Transparency
↓
Ranked Results
```

AI may interpret natural language, but the final ranking should rely on structured application data.

42. AI Prompt Architecture

Prompts should not be embedded throughout application logic.

Use:

```
promptBuilder
  ↓
structured context
  ↓
specific prompt
  ↓
LLM
```

A prompt should contain:

- task
 - structured input
 - output schema
 - safety instructions
 - uncertainty instructions
-

43. AI Output Rule

AI should return predictable structures.

Example:

```
{
  summary: "...",
  detectedConditions: [],
  relevantSpecialties: [],
  recommendedTreatments: [],
  confidence: 0.87
}
```

Bill:

```
{
  totalAmount: 205000,
  categories: [],
  lineItems: [],
  anomalies: [],
  summary: "...",
  confidence: 0.91
}
```

The frontend then decides how to display this structure.

44. AI Safety Boundary

OpenHealth should not allow an LLM to silently make medical diagnoses or fabricate hospital/coverage information.

The system should distinguish:

EXTRACTED
CALCULATED
PREDICTED
AI-GENERATED
VERIFIED

These labels should be preserved throughout the application.

45. Database Architecture

Database scripts remain independent of the Node application.

```
database/  
├── schema.sql  
├── seed.sql  
├── indexes.sql  
├── policies.sql  
└── README.md
```

46. Database Creation Order

Create tables according to dependencies:

```
profiles  
  ↓  
patient_profiles  
hospital_users  
  ↓  
hospitals  
  ↓  
departments  
doctors  
treatments  
  ↓  
hospital_treatments  
  ↓  
treatment_packages  
  ↓  
bed_types  
hospital_beds  
  ↓  
bed_reservations  
  ↓  
bookings
```

```
patient_profiles
  ↓
medical_documents
  ↓
report_analyses
bills
  ↓
bill_line_items
  ↓
bill_analyses
bill_shock_records

insurance_providers
insurance_policies

government_schemes
eligibility_checks

transparency_scores
audit_logs
```

47. Database Integrity

Use:

- foreign keys
- unique constraints
- check constraints
- indexes
- timestamps
- nullable fields only where meaningful

Example:

```
occupied_beds >= 0
reserved_beds >= 0
available_beds >= 0
```

48. Storage Architecture

Documents should not be stored directly inside PostgreSQL.

Use:

```
Supabase Storage
  ↓
medical-documents/
bills/
reports/
```

hospital-documents/

Database stores only metadata and secure references.

49. Authentication Architecture

```
React
  ↓
Supabase Auth
  ↓
Session
  ↓
JWT
  ↓
Express
  ↓
Authentication Middleware
  ↓
Role Middleware
  ↓
Service
  ↓
Database
```

50. Authorization Architecture

Frontend routing provides the first UX-level restriction.

Backend provides the actual security restriction.

Database RLS provides another enforcement layer.

```
UI restriction
  +
API authorization
  +
Database RLS
```

Never rely on the frontend alone.

51. Patient Data Isolation

A patient should be able to access:

their profile

their documents
their bills
their bookings
their insurance information

A patient should not be able to access another patient's documents.

52. Hospital Data Isolation

Hospital users should only modify records associated with their hospital.

Example:

Hospital A Staff
↓
Hospital A Beds
Hospital A Doctors
Hospital A Packages
Hospital A Bookings

They must not modify Hospital B.

53. Admin Access

Platform administrators have broader visibility but sensitive operations should still be logged.

Examples:

hospital verification
doctor verification
scheme updates
insurance updates
user management

54. Audit Logging

Important operations should create audit records.

Example:

Admin verified hospital
Hospital staff changed ICU availability
Patient uploaded bill
Patient created reservation

Admin modified scheme rules

This should be visible through the admin audit interface.

55. API Response Architecture

All APIs should follow a consistent pattern.

Success

```
{
  "success": true,
  "data": {}
}
```

Error

```
{
  "success": false,
  "message": "Hospital not found",
  "errorCode": "HOSPITAL_NOT_FOUND"
}
```

Never expose internal stack traces.

56. API Authentication Flow

For protected endpoints:

```
Request
↓
JWT
↓
authMiddleware
↓
Decode / Verify
↓
Load User Context
↓
roleMiddleware
↓
Controller
```

57. Search Request Flow

```
SearchBar.jsx
↓
searchService.js
↓
GET /api/search
↓
searchController
↓
searchService
↓
matchingService
↓
rankingService
↓
PostgreSQL
↓
Results
↓
SearchResults.jsx
```

58. Hospital Detail Request Flow

```
HospitalDetails.jsx
↓
hospitalService.getHospital()
↓
GET /api/hospitals/:id
↓
hospitalController
↓
hospitalService
↓
Database
↓
Hospital
Departments
Doctors
Treatments
Packages
↓
Frontend
```

59. Live Bed Request Flow

```
BedSearch.jsx
↓
bedService.js
↓
GET /api/beds/search
↓
bedController
↓
availabilityService
```

↓
Hospital Beds
↓
Availability
↓
Distance / Filters
↓
Results

60. Bed Reservation Request Flow

Reserve button
↓
bookingService.js
↓
POST /api/beds/reservations
↓
reservationController
↓
reservationService
↓
Check availability
↓
Create temporary hold
↓
Payment / deposit
↓
Reservation record
↓
Confirmation

Availability must be checked server-side immediately before creating a reservation.

61. Cost Prediction Request Flow

CostPrediction.jsx
↓
costService.js
↓
POST /api/cost/predict
↓
costController
↓
costPredictionService
↓
Pricing Data
+
Treatment
+
Hospital
+

Other Inputs
↓
Prediction
↓
Database
↓
Frontend

62. Report Analysis Request Flow

ReportAnalysis.jsx
↓
documentService.js
↓
POST /api/documents/:id/analyze
↓
documentController
↓
reportAnalysisService
↓
FastAPI
↓
OCR
↓
AI
↓
Structured Result
↓
Database
↓
React

63. Bill Analysis Request Flow

BillAnalysis.jsx
↓
billService.js
↓
POST /api/bills/:id/analyze
↓
billController
↓
billAnalysisService
↓
FastAPI
↓
OCR
↓
Line Item Extraction
↓
Anomaly Analysis
↓
Node

↓
Bill Shock Service
↓
Database
↓
React

64. Transparency Calculation Flow

Hospital Data
+
Package Data
+
Verification
+
Availability Freshness
+
Billing History
↓
transparencyService
↓
scoreService
↓
Transparency Score
↓
Database
↓
Hospital Page

65. Smart Matching Flow

Natural-language patient request
↓
AI / matchingService
↓
Department
↓
Treatment
↓
Hospital capability
↓
Location
↓
Availability
↓
Insurance / Scheme
↓
Transparency
↓
Ranking

66. Development Dependency Graph

The implementation should follow dependencies.

```
AUTH
↓
BASE LAYOUT
↓
DATABASE
↓
HOSPITAL DATA
↓
SEARCH
↓
HOSPITAL DETAILS
↓
BEDS
↓
BOOKINGS
↓
COST
↓
INSURANCE
↓
DOCUMENTS
↓
AI
↓
TRANSPARENCY
↓
ADMIN
```

Not every feature must strictly follow this order, but dependencies should be respected.

67. Hackathon Development Phase 1

Foundation

Implement:

```
Repository
React + Vite
Tailwind CSS
React Router
Express
Supabase
Environment configuration
Authentication
Global layout
Navigation
```

Deliverable:

User can log in and reach a functional application shell.

68. Phase 2 — Healthcare Discovery

Build:

Hospitals
Departments
Doctors
Treatments
Search
Filters
Hospital detail
Comparison

Deliverable:

User can describe a healthcare need and compare hospital options.

69. Phase 3 — Bed Intelligence

Build:

Bed types
Hospital beds
Availability
Emergency search
Reservation

Deliverable:

User can identify available beds and initiate a reservation.

70. Phase 4 — Cost & Coverage

Build:

Treatment packages
Cost estimation
Insurance

Government schemes
Eligibility
Patient contribution

Deliverable:

User can understand expected financial responsibility before treatment.

71. Phase 5 — AI Documents

Build:

Document upload
OCR
Report analysis
Bill extraction
Bill analysis
AI explanations

Deliverable:

User can upload a document and receive structured intelligence.

72. Phase 6 — Transparency

Build:

Transparency score
Bill Shock Index
Hospital transparency page
Billing consistency

Deliverable:

User can evaluate the transparency of hospital options.

73. Phase 7 — Hospital Portal

Build:

Hospital dashboard
Beds

Doctors
Departments
Treatments
Packages
Bookings
Transparency

Deliverable:

Hospital can maintain its own operational information.

74. Phase 8 — Admin

Build:

Verification
Users
Hospitals
Doctors
Schemes
Insurance
Analytics
Audit logs

Deliverable:

Platform administrators can maintain data quality and governance.

75. Phase 9 — Polish

Focus on:

Responsive layouts
Loading states
Empty states
Error states
Toasts
Animations
Accessibility
Charts
Map integration
Demo data

76. MVP Core Files

Frontend

App.jsx

pages/patient/Dashboard.jsx
pages/patient/Search.jsx
pages/patient/SearchResults.jsx
pages/patient/HospitalDetails.jsx
pages/patient/CompareHospitals.jsx
pages/patient/Beds.jsx
pages/patient/Emergency.jsx
pages/patient/CostPrediction.jsx
pages/patient/ReportAnalysis.jsx
pages/patient/BillAnalysis.jsx
pages/patient/InsuranceChecker.jsx
pages/patient/Bookings.jsx

components/search/SearchBar.jsx
components/hospitals/HospitalCard.jsx
components/hospitals/HospitalComparison.jsx
components/beds/BedAvailabilityCard.jsx
components/emergency/EmergencySearch.jsx
components/bills/BillAnalysis.jsx
components/documents/DocumentUploader.jsx
components/costs/CostRangeCard.jsx
components/transparency/TransparencyScore.jsx

77. MVP Backend Files

controllers/hospitalController.js
controllers/searchController.js
controllers/bedController.js
controllers/bookingController.js
controllers/documentController.js
controllers/billController.js
controllers/costController.js
controllers/insuranceController.js
controllers/transparencyController.js

services/search/searchService.js
services/search/matchingService.js
services/search/rankingService.js

services/hospitals/hospitalService.js
services/beds/availabilityService.js
services/bookings/reservationService.js
services/bills/billAnalysisService.js
services/costs/costPredictionService.js
services/transparency/scoreService.js

78. MVP AI Files

```
ai-service/app/main.py
```

```
ai-service/app/services/ocr_service.py  
ai-service/app/services/report_analyzer.py  
ai-service/app/services/bill_analyzer.py  
ai-service/app/services/matching_service.py
```

79. MVP Database Files

```
database/schema.sql  
database/seed.sql
```

The initial schema should prioritize:

```
profiles  
patient_profiles  
hospitals  
departments  
doctors  
treatments  
hospital_treatments  
treatment_packages  
bed_types  
hospital_beds  
bed_reservations  
bookings  
medical_documents  
report_analyses  
bills  
bill_line_items  
bill_analyses  
bill_shock_records  
insurance_providers  
government_schemes  
eligibility_checks  
transparency_scores  
audit_logs
```

80. Demo-First Strategy

The hackathon implementation should not wait for every hospital to provide a real-time API.

Use seeded demo data for:

- hospitals
- doctors
- departments
- treatments
- packages
- beds

- insurance
- schemes
- previous bills
- transparency scores

Then clearly label live operational data according to its actual source and freshness.

81. Primary Demonstration Flow

The team should be able to demonstrate:

```
Patient Login
  ↓
"Knee replacement near me"
  ↓
Smart Hospital Search
  ↓
Compare 3 Hospitals
  ↓
View Cost Range
  ↓
Check Insurance
  ↓
View Transparency Score
  ↓
Check ICU / Bed Availability
  ↓
Reserve
  ↓
Upload Medical Report
  ↓
AI Analysis
  ↓
Upload Medical Bill
  ↓
AI Bill Analysis
  ↓
Bill Shock Index
```

This should be treated as the **golden path**.

82. Root Package Structure

The root `package.json` should provide convenient scripts.

Conceptually:

```
{
  "scripts": {
    "client": "npm --prefix client run dev",
    "server": "npm --prefix server run dev",
    "dev": "concurrently \"npm run client\" \"npm run server\""
  }
}
```

The AI service can be started independently during development.

83. Environment Variables

Root `.env.example`:

```
SUPABASE_URL=
SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=

DATABASE_URL=

JWT_SECRET=

AI_SERVICE_URL=
LLM_API_KEY=

MAPS_API_KEY=

PAYMENT_KEY=
PAYMENT_SECRET=
```

Never commit actual secrets.

84. Frontend Environment

```
VITE_SUPABASE_URL=
VITE_SUPABASE_ANON_KEY=
VITE_API_BASE_URL=
VITE_MAPS_API_KEY=
```

The frontend must never contain service-role secrets.

85. Backend Environment

```
SUPABASE_URL=
```

SUPABASE_SERVICE_ROLE_KEY=
DATABASE_URL=

AI_SERVICE_URL=
LLM_API_KEY=

MAPS_API_KEY=

PAYMENT_KEY=
PAYMENT_SECRET=

86. Engineering Rules

Rule 1

Pages represent screens.

Rule 2

Components represent reusable UI.

Rule 3

Services represent application/business operations.

Rule 4

Controllers handle HTTP requests.

Rule 5

Routes define API endpoints.

Rule 6

Middleware handles cross-cutting concerns.

Rule 7

Database schema defines persistence.

Rule 8

AI service handles AI-heavy operations.

Rule 9

Sensitive logic must never depend exclusively on the frontend.

Rule 10

Do not introduce TypeScript.

87. What Should NOT Be Done

Do not create:

```
everything.js  
hospitalEverything.jsx  
massive Dashboard.jsx
```

Do not:

- place SQL inside React components
 - place API calls inside every component
 - place AI prompts inside controllers
 - place authorization only in the frontend
 - duplicate hospital-card logic across pages
 - mix database operations with route definitions
 - hard-code secrets
 - use AI for deterministic arithmetic when application logic can perform it
-

88. Recommended Service Boundaries

A useful rule is:

```
Hospital Service  
→ hospital information
```

```
Search Service  
→ discover and rank
```

```
Bed Service  
→ availability
```

```
Booking Service  
→ reservations
```

```
Cost Service  
→ financial estimation
```

```
Insurance Service  
→ coverage
```

Document Service
→ file lifecycle

AI Service
→ interpretation

Bill Service
→ billing intelligence

Transparency Service
→ transparency scoring

This prevents the backend from becoming one large service.

89. Request Lifecycle Standard

Every authenticated request should conceptually follow:

```
Request
↓
CORS
↓
Rate Limit
↓
Authentication
↓
Authorization
↓
Validation
↓
Controller
↓
Business Service
↓
Database / External Service
↓
Response Formatter
↓
Client
```

90. Frontend Data Lifecycle

```
Page
↓
Hook
↓
Service
↓
API
↓
```


Response
↓
Hook state
↓
Component
↓
UI

Example:

HospitalDetails.jsx
↓
useHospital()
↓
hospitalService.getHospital()
↓
GET /api/hospitals/:id
↓
Hospital Data
↓
HospitalProfile
HospitalDoctors
HospitalPackages
HospitalAvailability
HospitalTransparency

91. Error Handling Strategy

Errors should be handled at multiple levels.

API

Returns structured error.

Service

Normalizes API error.

Hook

Maintains error state.

Component

Displays user-friendly message.

Example:

API Error
↓

```
"HOSPITAL_NOT_FOUND"
```

↓

```
"No hospital matching this request was found."
```

Never display raw server errors.

92. Logging Strategy

Backend should log:

- authentication failures
- unexpected server errors
- AI processing failures
- payment failures
- booking errors
- database failures
- suspicious activity

Do not log sensitive medical document contents.

93. Performance Rules

Frontend

- lazy-load large pages
- optimize images
- paginate long lists
- avoid unnecessary rerenders
- cache frequently accessed search results
- debounce search input

Backend

- paginate database results
- index searchable columns
- avoid unnecessary joins
- cache suitable read-heavy information

AI

- process asynchronously where necessary
- limit document size

- reject unsupported files early
 - avoid repeated AI calls
-

94. File Upload Rules

Before processing:

```
File type check
↓
File size check
↓
Authentication check
↓
Secure storage
↓
PII handling
↓
AI processing
```

Supported MVP types can include:

```
PDF
JPG
JPEG
PNG
```

95. Booking Consistency Rule

A bed booking must always re-check current availability on the server.

Flow:

```
Client sees 3 beds
↓
User clicks reserve
↓
Server checks current availability
↓
If available → create hold
If unavailable → return failure
```

The client-side displayed value cannot be treated as authoritative.

96. AI Processing Status

Documents should have lifecycle states.

uploaded
processing
processed
failed

Frontend:

Uploaded ✓
Processing •
Analysis ○
Complete ○

97. Recommended Database Seed

The seed file should contain a realistic demo environment.

3-5 Hospitals
10+ Departments
15+ Doctors
10+ Treatments
10+ Packages
Multiple Bed Types
Different Bed Availability
2-3 Insurance Providers
2-3 Government Schemes
Sample Patients
Sample Reports
Sample Bills
Transparency Scores
Historical Bill Shock Records

This makes the application immediately demoable.

98. Deployment

Frontend

React + Vite
↓
Vercel

Backend

Node + Express
↓

Cloud deployment

AI

FastAPI

↓
Docker / cloud deployment

Database

Supabase PostgreSQL

Storage

Supabase Storage

99. Docker Structure

Optional for the hackathon but useful for reproducibility:

```
OpenHealth/  
├── client/  
│   └── Dockerfile  
├── server/  
│   └── Dockerfile  
└── ai-service/  
    └── Dockerfile
```

Do not introduce Docker complexity unless it improves deployment or team consistency.

100. Documentation Structure

```
docs/  
├── prd.md  
├── architecture.md  
├── frontend.md  
├── database.md  
├── api.md  
├── ai.md  
├── security.md  
├── design.md  
├── rules.md  
└── phases.md
```

During active coding:

memory.md

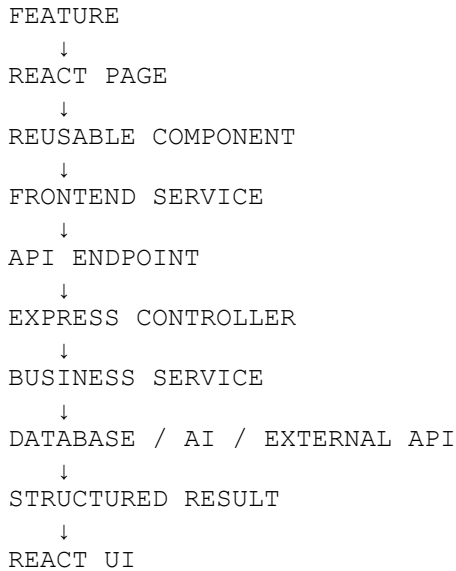
should track implementation state.

101. Final Codebase Map

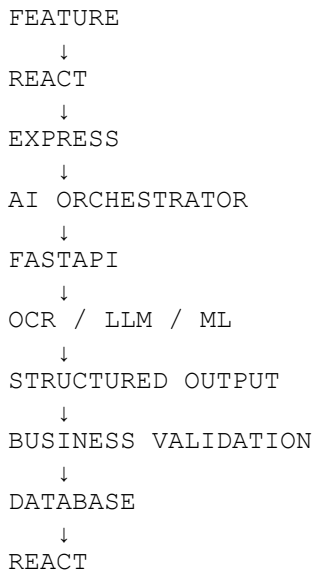
```
OpenHealth/
├── client/
│   └── React + Tailwind
│       ├── pages
│       ├── components
│       ├── services
│       ├── hooks
│       ├── context
│       ├── routes
│       └── utils
├── server/
│   └── Node + Express
│       ├── routes
│       ├── controllers
│       ├── services
│       ├── middleware
│       ├── validators
│       └── config
├── ai-service/
│   └── Python + FastAPI
│       ├── OCR
│       ├── report analysis
│       ├── bill analysis
│       ├── matching
│       └── LLM integration
├── database/
│   ├── schema.sql
│   ├── seed.sql
│   ├── indexes.sql
│   └── policies.sql
└── docs/
    ├── prd.md
    ├── architecture.md
    ├── frontend.md
    ├── database.md
    ├── api.md
    ├── ai.md
    ├── security.md
    ├── rules.md
    └── phases.md
```

102. Final Architecture Principle

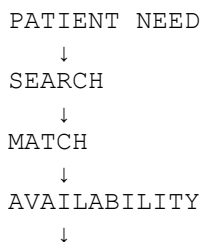
The complete OpenHealth architecture should remain traceable:



For AI-heavy workflows:



For healthcare resource workflows:



COST
↓
COVERAGE
↓
TRANSPARENCY
↓
RESERVATION

The architecture therefore keeps **React + Tailwind CSS** responsible for the interface, **Express** responsible for application APIs and orchestration, **Supabase/PostgreSQL** responsible for persistent healthcare data, and the **FastAPI AI service** responsible for OCR and AI-heavy processing. This separation gives OpenHealth a structure that is practical for the hackathon while leaving a clear path for future hospital, insurance, payment, and healthcare-network integrations. The submitted project concept also explicitly positions React/Tailwind, Node/Express, Supabase/PostgreSQL, Python/FastAPI, OCR, LLM APIs, Google Maps, and Razorpay/UPI within these corresponding technical layers.

Document 4 complete — Software Architecture & Development Structure.